CROSS / PROBLEM 04

Problem 4: Private Qwen3-0.6B inference with CROSS

CADathlon · `agent_pset` · AI agents allowed · 100 points · proposed duration: 8 hours

1. Introduction

A client wants to use a language model without exposing its prompt to the inference server. CKKS homomorphic encryption allows the server to compute on encrypted values. CROSS maps HE arithmetic to AI accelerators using Basis-Aligned Transformation (BAT) and Memory-Aligned Transformation (MAT). Your task is to extend CROSS to run **Qwen/Qwen3-0.6B**, including all 28 decoder layers and the vocabulary projection, on encrypted activations.

Coding agents are permitted for this track. Keep a short record of tools, models, delegated tasks and human verification in `REPORT.md`. Only public fixtures may be sent to external services. The provided CROSS code, tutorials and public numerical references are available to every team.

This is a staged engineering challenge. The full model is the final target; operator and block milestones earn meaningful partial credit. CROSS currently supports the CNN compiler path, not automatic compilation of this transformer. A reduced-width model or a single encrypted layer earns only its milestone credit, never the full-model points.

2. Problem statement

2.1 Fixed model and workload

Use the immutable model revision in `test/model_source.json`, the accompanying `qwen3_config.json`, and organizer-supplied checkpoint and tokenizer files. Keep every learned tensor unchanged. Cast weights for computation as needed; do not prune, retrain, distill, remove layers or reduce the vocabulary.

Property	Required value
Decoder layers / hidden width / MLP intermediate width	28 / 1024 / 3072
Query heads / key-value heads / head dimension	16 / 8 / 128
Q, K, V projection widths	2048, 1024, 1024
Vocabulary / embedding and output weights	151936 / tied
Normalization / MLP	RMSNorm, including Q/K normalization / SiLU-gated MLP

Property	Required value
RMSNorm epsilon / RoPE base	1e-6 / 1000000
Ranked batch size	1
Prefill buckets	8, 16 and 32 token positions
Generation	4 additional greedy tokens from a 16-position prompt

These dimensions come from the official pinned configuration. In particular, Q width is not equal to hidden width. Use head dimension 128, not `1024 // 16`. Repeat each KV head for its two associated Q heads.

Tokenization and embedding lookup happen on the trusted client. Prefill tests may use raw text tokenized without a chat template, so the eight-token bucket is meaningful. If a chat template is used for generation, set `enable_thinking=False`. Organizers freeze the resulting token IDs and never silently truncate a formatted conversation to fit a bucket. Greedy selection is a deterministic contest protocol, not a claim about optimal Qwen generation quality. The client encrypts the resulting embeddings. The server returns encrypted last-position logits over **all 151936 tokens**. The client decrypts and takes argmax (smallest token ID breaks an exact tie).

For four-token generation, repeated encrypted prefill is allowed, as is an encrypted KV cache. Each call includes the previously selected client tokens as encrypted embeddings; no generated token IDs or cached plaintext K/V are sent to the server. Fixed-size buckets, public position indices, causal masks, and declared padding masks are allowed. Length and mask leakage are part of this threat model. No secret-dependent branch, mask, index or memory access may reveal token content. Left padding, if used, has position IDs assigned only to real tokens; organizers supply those IDs. Public smoke fixtures have no padding and position IDs `0..S-1`.

2.2 Trust and arithmetic contract

The server is honest-but-curious. Model weights are public to the evaluator; model secrecy, traffic-analysis resistance, malicious-server verification and multi-party key sharing are out of scope. The client alone owns the secret key. The evaluator receives ciphertexts, public model constants, public and evaluation keys, and shape metadata. It must not receive plaintext embeddings, token IDs, hidden states, attention scores, logits or the secret key.

All transformer computation after embedding lookup runs under HE: input and post-attention RMSNorm, Q/K/V projections, per-head Q/K RMSNorm, RoPE, causal grouped-query attention, output projection, both residual connections, the gated SiLU MLP, final RMSNorm and vocabulary projection. RoPE factors and causal masks are public constants. Attention scores and softmax denominators are encrypted. Apply a public causal mask to approximate exponentials before normalizing; do not send a numerical approximation through an artificial negative-infinity mask. Intermediate decryption/re-encryption by the client, plaintext server fallbacks and remote plaintext LLM calls are forbidden. Bootstrapping performed under encryption is allowed.

CKKS approximates arithmetic. Supply an explicit polynomial/iteration design for SiLU, exponential/normalization and reciprocal square root, with intervals, coefficients, degree/iteration count and error evidence from public calibration data. Plaintext `exp`, `sqrt`, clipping or max-subtraction on secret activations is not an HE

implementation. Avoid evaluating the exact plaintext Qwen with the frontend's default SiLU-to-square substitution: it changes the model. Judge both CKKS error against your declared approximate circuit and total approximation error against original Qwen.

Use CROSS's CKKS evaluator and canonical `Polynomial` values, whose payload is `(batch, num_elements, r, c, num_moduli)` with `r*c == degree`. Lists/tuples of these values may represent tiled tensors. Maintain a 128-bit security target with the actual Q/P chains, degree, distributions and key-switch parameters; use the repository's validated security planning and report any extension. An insecure `packing.test_only_parameters(...)` run is only a local diagnostic. Ranked cases must pass the organizer's parameter review. Preserve secure encryption randomness; fixture seeds control data and weights, never noise.

2.3 Files and milestones

Implement `create_session` in `src/qwen_he.py`. It returns an object with:

```
encrypt(embeddings)          # client; float64 array (B,S,H) -> ciphertext tree
evaluate(ciphertexts)        # server; ciphertext tree -> encrypted (B,V) logits
decrypt(ciphertexts)         # client; encrypted logits -> float64 array (B,V)
close()                     # release keys, mappings and device buffers
```

The constructor receives `config`, a dictionary of named plaintext weight arrays, `batch`, `sequence_length` and `devices`. For smoke cases there is no padding or cache. Extend the interface in an accompanying module for explicit position IDs, padding and decode state; document that extension. The trusted harness holds the session for testing. Your report must separately describe client/server ownership; a single Python object containing a secret key is not evidence of process isolation. For full privacy credit, export a secret-key-free evaluator and demonstrate it in a separate process.

For partial-credit judging, the same file declares `create_operator_session(operation, config, weights, *, batch, sequence_length, devices)` and `create_block_session(config, weights, *, layer_index, batch, sequence_length, devices)`. Both return the same four-method lifecycle. The block entrypoint accepts and returns `(B,S,H)` tensors and includes both residuals, before the final model normalization/head. It is tested at `layer_index=0` with full checkpoint weights. The operator entrypoint uses the following public contract (H=1024, D=128):

Operator name	Client operands → decrypted result	Plaintext weights
`rmsnorm`	`(B,S,H)` → `(B,S,H)`	`weight` : length H
`qk_rope`	tuple Q `(B,16,S,D)`, K `(B,8,S,D)` → normalized, rotated Q/K tuple	`q_norm.weight`, `k_norm.weight` : length D
`gqa`	tuple normalized/rotated Q, K and V `(B,8,S,D)` → attended `(B,S,2048)` before output projection	empty dictionary
`gated_mlp`	post-norm `(B,S,H)` → `(B,S,H)`	`gate_proj.weight`, `up_proj.weight`, `down_proj.weight`

Operator name	Client operands → decrypted result	Plaintext weights
`linear`	`(B,S,K)` → `(B,S,M)` for supplied matrix	`weight` : `(M,K)` ; Q1 includes a Q projection and the full vocabulary projection

For tuples, `encrypt` / `decrypt` preserve logical tuple order; `evaluate` only receives ciphertext trees. Position IDs are `0..S-1` in Q1. Operator weights come from layer 0 (or the tied embedding matrix for the vocabulary projection). Only the organizer's trusted client compares decrypted tensors with references.

Add helpers under `src/` and submit any necessary CROSS changes as `cross.patch`. Likely implementation sites are `nn.py`, `packing.py`, `mapping.py`, `bsgs.py`, `ckks_ctx.py` and `bootstrapping.py`. Changes must preserve the public ciphertext contract. Either extend the canonical compiler or use CROSS's documented direct context evaluator with a complete rotation, level and bootstrap plan. CROSS does not currently expose automatic bootstrap insertion through the public frontend. Plan refreshes explicitly; model depth does not fit merely by choosing a larger array shape.

Milestone	Required artifact and evidence
A1: inspect and plan	Reproduce public numerical fixtures; export full model weights; tensor/layout map; ciphertext count, rotations, scales and depth schedule; client/server diagram
A2: encrypted operators	Real encrypted RMSNorm, Q/K normalization + RoPE, GQA attention, SiLU-gated MLP and tiled linear projection; public and fresh unseen inputs
A3: decoder block	Complete first Qwen decoder block at width 1024, including residuals and all nonlinearities; encrypted input to encrypted output
A4: complete prefill	All 28 blocks, final normalization and full vocabulary logits for every prefill bucket
A5: generation	Four successive generated tokens, with encrypted state or encrypted recomputation

2.4 Tests and numeric gates

`test/test1.in` through `test6.in` are deterministic, small decoder fixtures. Their `.out` files hold actual last-position logits from the exact numerical reference. They cover one/multiple layers, non-square projection dimensions, grouped-query heads, multiple token lengths, residuals, RoPE and tied output weights. `src/reference.py` is a readable, float64 semantic guide. These are **toy correctness cases**, not evidence that the 0.6B model ran. `src/qwen_he.py.golden` is a plaintext oracle for those same semantics and is excluded from the contestant release; it is not an HE solution or a latency baseline.

Organizers additionally freeze six full-size suites before the event:

Suite	Fixed evaluation workload
Q1	Operator suite, Qwen dimensions, 8 public calibration tensors plus 8 hidden tensors per operator
Q2	Decoder layer 0, S=8, 8 hidden embedding sequences
Q3	Full model S=8, 16 hidden prompts
Q4	Full model S=16, 16 hidden prompts
Q5	Full model S=32, 16 hidden prompts
Q6	Full model S=16, 8 hidden prompts, 4 generated tokens each

Prompts, checkpoints and calibration inputs are distinct from random smoke weights. Do not specialize to supplied sample IDs or read reference outputs. For each finite tensor define `nRMSE = RMS(y-ref) / max(RMS(ref), 0.01)`, where `RMS(z) = sqrt(mean(z*z))`. The denominator floor makes zero/near-zero reference tensors testable with approximate encryption. HE versus the declared plaintext approximate circuit must have nRMSE ≤ 0.01 and maximum absolute error ≤ 0.10. The approximate circuit versus original Qwen must have nRMSE ≤ 0.05 for operator/block outputs and ≤ 0.10 for full logits. For full prefill, at least 90% of prompts across Q3–Q5 must match the original Qwen argmax; report each suite separately. For decode, at least 90% of 32 teacher-forced next-token decisions must agree, and at least 6/8 complete four-token greedy sequences must match in free-running generation. Both are required. Teacher-forced contexts use the original model's preceding tokens; free-running contexts use the submission's preceding tokens. Publish all errors and top-1 rates, including failures. These are proposed gates for organizer calibration; they are frozen before teams receive hidden cases.

The bundled runner checks encrypted outputs against the exact toy oracle with nRMSE ≤ 0.10 and max error ≤ 0.20. It checks finite values, shapes and ciphertext wrappers. It is a local development check, not the hidden-suite judge, a security proof or an automated audit of evaluator internals.

3. How to compile and run

Use the organizer environment containing the pinned CROSS checkout, NumPy, PyTorch, JAX and its accelerator backend. Do not upgrade packages during the contest. `CROSS_ROOT` defaults to the containing repository and can be set to an alternate checkout. A CPU is sufficient for numerical-reference tests.

```
cd cadathlon/agent_pset/p4_qwen/src
make all
make reference-test               # author package: exact numerical oracle
make test                         # your HE implementation; starts with TODO
python runner.py --case ../test/test1.in
make submit                       # writes ../submission.tar.gz
```

`make test` emits one JSON result per case and exits nonzero on any failure. Debug logging goes to stderr. Public `.out` comparison uses tolerances rather than bytewise `diff`, since CKKS encryption is randomized. Never report the plaintext reference run as an encrypted result.

Proposed judging allocation: one TPU v6e chip, fixed host and package image per run; sequential exclusive reservations. CPU fallback is for development and cannot be compared to TPU leaderboard times. Build/key generation limit: **60 minutes per shape**, separate from inference; **600 seconds per prompt** and **2400 seconds per four-token sequence**. Cap host RSS at **128 GiB**; device allocation must fit the assigned chip. OOM/timeout is a failed case. These ceilings are contest budgets, not measured Qwen feasibility claims.

Time preparation separately from online inference. For each shape perform one untimed warm-up with an unrelated fresh input, then measure each scored input once; report median and p95 over that suite. Before starting and ending a device timer, synchronize all output ciphertext leaves using `jax.block_until_ready`. Also measure full client encryption, transfer, server evaluation, return and client decryption. Online rotations, refreshes and memory transfers count. Keys, public constants and compiled executables may be reused; prompt-specific results and intermediate activations may not. Report setup seconds, warm server seconds, end-to-end seconds, peak host/device memory, bytes transferred, ciphertext/rotation/bootstrap counts and errors.

4. How to score

Each row is awarded only for observed evidence. Hidden tests use the same published contract. Failed shape, nonfinite output, timeout or invalid cryptographic parameters gives zero for that case. Server plaintext exposure or intermediate decryption invalidates all encrypted-computation points.

Component	Points	Rule
A1: model and execution plan	10	2 each: reproducible oracle, correct dimensions/weights, packing/rotation plan, level/refresh plan, trust-boundary report
A2: encrypted operators	25	5 each: RMSNorm, Q/K normalization+RoPE, GQA+causal normalization, gated SiLU MLP, tiled projection; points multiplied by passing hidden-input fraction
A3: full-width decoder block	15	15 × passing fraction of Q2
A4: all-layer prefill	30	10 per Q3/Q4/Q5 × fraction passing both numeric gates; aggregate top-1 gate must also pass
A5: four-token generation	10	10 if both decode agreement gates and every step's numeric gates pass; otherwise 0
Efficiency	5	Only if A4 earns 30: `5 * min(1, 60 / T)` where T is median warm server seconds over Q3–Q5

Component	Points	Rule
Reproducibility	5	1 each: executable submission, versions/checkpoint hashes, timing/ memory evidence, regression tests, AI-use log

The 60-second efficiency target is a scoring constant, not an existing CROSS Qwen baseline. Numeric-error checks and code audit both apply: matching outputs alone does not prove HE. Report actual completed milestones; a written plan cannot earn implementation points. Ties break by A4 score, then A5 score, then lower full-model end-to-end median, then lower peak device memory.

Submit `src/`, `cross.patch` (empty if unchanged), `REPORT.md`, raw measurement JSON and environment/ checkpoint hashes. Do not include secret keys, model weights, datasets, reference outputs, caches or hidden tests. A six-page report is sufficient; logs and source code do not count toward this limit.

5. References

1. Tong et al., Leveraging ASIC AI Chips for Homomorphic Encryption, HPCA
2. Supplied `CROSS_paper.pdf`; paper.
3. CROSS tutorial: Easy HE Background, CROSS, TPU Novice to Master, NTT Algorithms, Open Challenge.
4. CROSS `jaxite_word/API_REFERENCE.md`, `bootstrapping_test.py`, `bsgs_test.py`, `mapping_compile_test.py` and the canonical demo code.
5. Qwen3-0.6B model card and the pinned configuration cited above. Use the provided checkpoint revision throughout.